# BellaStyle Manager

Sistema de gestión para salón de belleza

---

## Manual Técnico

Arquitectura, base de datos, código fuente e instalación

**César Pestana**
Proyecto de Aula · Ingeniería de Sistemas
Fundación Universitaria Colombo Internacional
Cartagena de Indias, noviembre de 2026 · Versión 4.0

# Contenido

**Introducción**

**2. Manual técnico**

**Glosario**

**Control del documento**

**Anexo A. Diagramas en tamaño completo**

# Introducción

Este documento es el **Manual Técnico** de **BellaStyle Manager**. Va dirigido al personal de soporte técnico o a quien haga sus veces, y detalla la estructura del sistema para entender su funcionamiento y facilitar su instalación y mantenimiento.

Sigue la estructura de la guía *Manuales técnicos de la aplicación* (SENA): alcance, descripción de los procesos, diagramas UML, administración de usuarios, modelo relacional de la base de datos, descripción de la plataforma, documentación del código fuente y acuerdos de niveles de servicio (ANS).

El sistema está publicado en **https://bellastyle.onrender.com**: el servidor corre en **Render** y la base de datos MySQL en **Aiven**. El código fuente está en el repositorio de GitHub `cesar-pestana26/bellastyle-manager`. En el computador de desarrollo el mismo sistema se ejecuta en `http://localhost:3000`.

# 2. Manual técnico

| No | Descripción | Detalle |
|---|---|---|
| **1** | Alcance | **Objetivo general:** desarrollar un sistema web que permita al salón BellaStyle gestionar sus citas, su personal y su catálogo de servicios y productos.<br>**Objetivos específicos cumplidos:**<br>• Registro e inicio de sesión con tres roles: Administradora, Estilista y Clienta, con contraseñas protegidas con **hash (bcrypt)**.<br>• Agendamiento de citas que muestra **solo las horas libres**, según el horario y las ausencias de cada estilista, con **asignación automática** de la estilista según servicio y carga de trabajo.<br>• Reprogramación de citas (**una sola vez** por cita), cancelación y cambio de estado (Pendiente, Confirmada, Finalizada, Cancelada), sin cruces de horario para la clienta.<br>• Gestión de servicios, productos, estilistas y clientas (incluida su profesión) por parte de la administradora, con **fotos** de servicios y productos.<br>• **Historial de servicios** de cada clienta con las notas de la estilista y **comprobante PDF** del servicio finalizado.<br>• **Calificación** de las citas finalizadas (1 a 5 estrellas) y **chat entre la clienta y su estilista** por cada cita.<br>• Personalización: **foto de perfil** para los tres roles y **vitrina** de trabajos del salón.<br>• Indicadores de ingresos (administradora) y de comisión del 40 % (estilista).<br>• **Chat de asesoría con Bella**, un asistente con **inteligencia artificial** (Gemini API) que responde cualquier pregunta con los datos reales del salón; si la IA falla, responde un asistente por palabras clave, y la administradora puede contestar desde su panel.<br>• **Publicación en la nube**: servidor en Render y base de datos MySQL en Aiven.<br>**Límites (no cubiertos):** pagos y facturación electrónica, proveedores, movimientos de inventario y notificaciones por correo. |
| **2** | Descripción de los procesos | El sistema apoya el proceso principal del salón, **gestionar citas**, y los procesos de apoyo de **gestión de usuarios**, **gestión del catálogo** y **atención por chat**. Los diagramas de casos de uso muestran qué hace cada actor (se dividen en dos para que se lean mejor): |

| No | Descripción | Detalle |
|---|---|---|

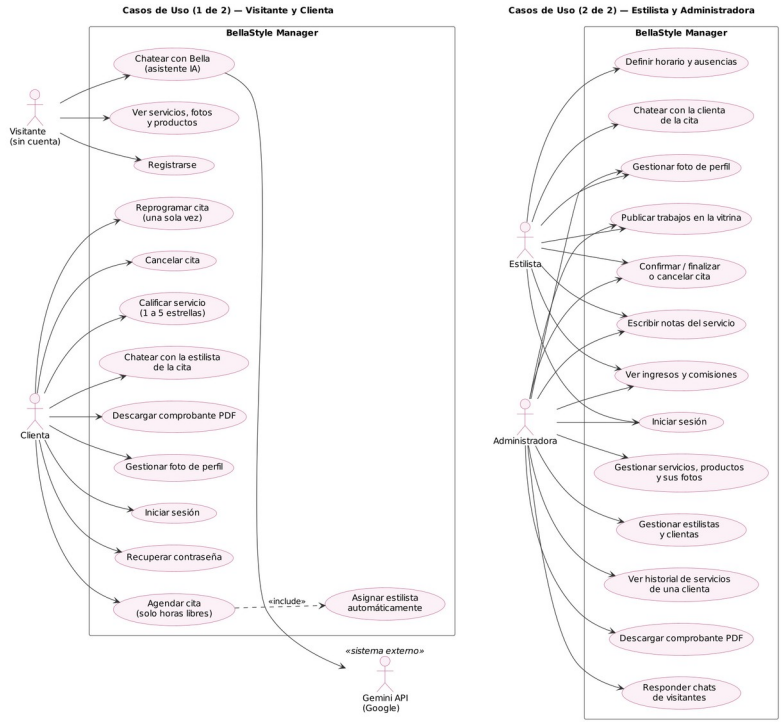



*Figura 1. Diagramas de casos de uso (1: visitante y clienta; 2: estilista y administradora)*

**Requerimientos funcionales:**

| Código | Requerimiento |
|---|---|
| RF01 | Registrar cuentas de clienta, estilista y administradora (con código). |
| RF02 | Iniciar sesión por rol y recuperar la contraseña. |
| RF03 | Agendar citas mostrando solo horas libres y validando horario de la estilista, ausencias, domingos, fechas pasadas y cruces. |
| RF04 | Asignar automáticamente la estilista libre con menos citas del día. |
| RF05 | Reprogramar (una sola vez por cita) y cancelar citas (clienta). |
| RF06 | Confirmar y finalizar citas (estilista y administradora). |
| RF07 | Crear, editar, activar y desactivar servicios y productos. |
| RF08 | Activar o desactivar estilistas y asignarles servicios. |
| RF09 | Editar datos de clientas, incluida su profesión. |
| RF10 | Mostrar ingresos y comisión (40 %) de las citas finalizadas. |
| RF11 | Permitir que un visitante sin cuenta escriba al salón por un chat, sin iniciar sesión. |
| RF12 | Responder con inteligencia artificial (Gemini API) cualquier |

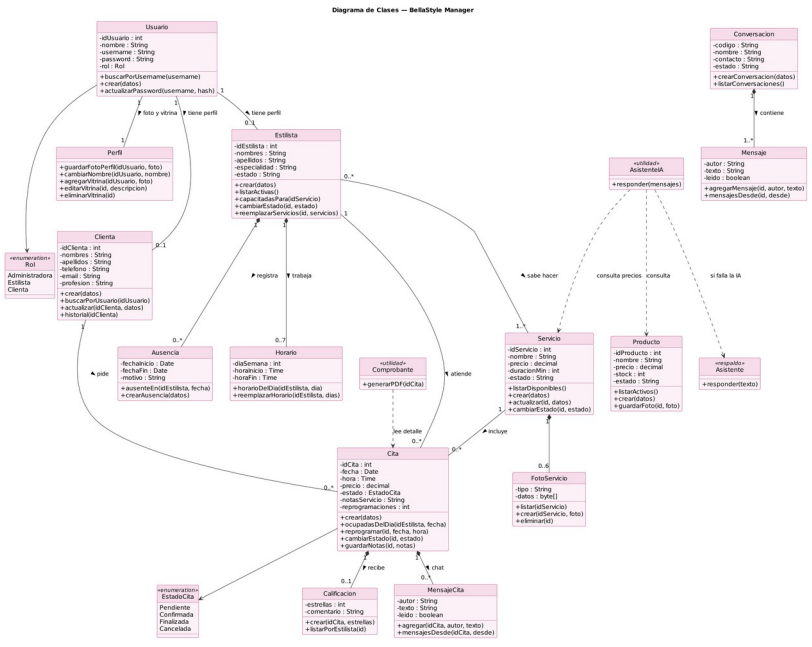
<table>
<tr><th>No</th><th>Descripción</th><th>Detalle</th></tr>
<tr><td></td><td></td><td>
<table>
<tr><td></td><td>pregunta del visitante usando los datos reales del salón; si la IA no está disponible, responder preguntas frecuentes por palabras clave.</td></tr>
<tr><td>RF13</td><td>Notificar mensajes nuevos (contadores) y permitir que la administradora responda y cierre conversaciones.</td></tr>
<tr><td>RF14</td><td>Definir el horario semanal y las ausencias de cada estilista.</td></tr>
<tr><td>RF15</td><td>Calificar con 1 a 5 estrellas las citas finalizadas.</td></tr>
<tr><td>RF16</td><td>Abrir un chat entre la clienta y la estilista por cada cita.</td></tr>
<tr><td>RF17</td><td>Registrar las notas del servicio y consultar el historial de servicios de cada clienta (estilista y administradora).</td></tr>
<tr><td>RF18</td><td>Generar el comprobante PDF de una cita finalizada.</td></tr>
<tr><td>RF19</td><td>Subir, cambiar y quitar fotos de servicios, productos, perfil y vitrina.</td></tr>
<tr><td>RF20</td><td>Validar teléfonos colombianos de 10 dígitos y correos electrónicos.</td></tr>
</table>
</td></tr>
<tr><td>3</td><td>Diagramas UML</td><td>Diagrama de clases. Representa las entidades del sistema, sus atributos, las operaciones de cada modelo y sus relaciones:<br> 

Figura 2. Diagrama de clases<br><br>Diagrama de secuencia – Agendar cita. Muestra cómo viaja la petición por las capas hasta la base de datos:</td></tr>
</table>

| No | Descripción | Detalle |
|---|---|---|

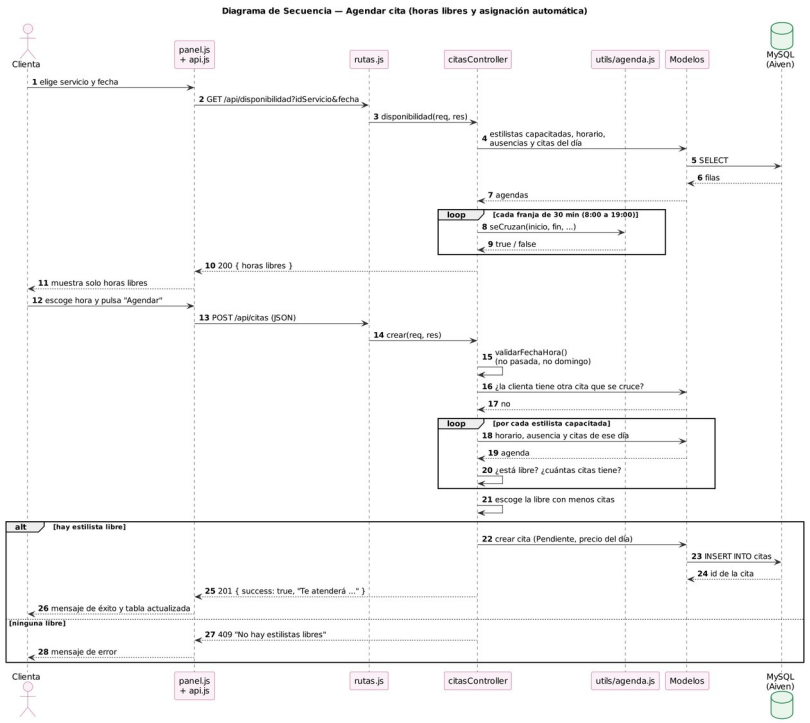



*Figura 3. Diagrama de secuencia: agendar cita*

**Diagrama de secuencia – Chat de asesoría con Bella (IA).** Muestra cómo el mensaje del visitante llega al servidor, cómo Bella prepara la respuesta con la API de Gemini (o con el respaldo de palabras clave) y cómo la administradora puede responder. Como el chat usa polling, cada navegador pregunta cada pocos segundos si hay mensajes nuevos:

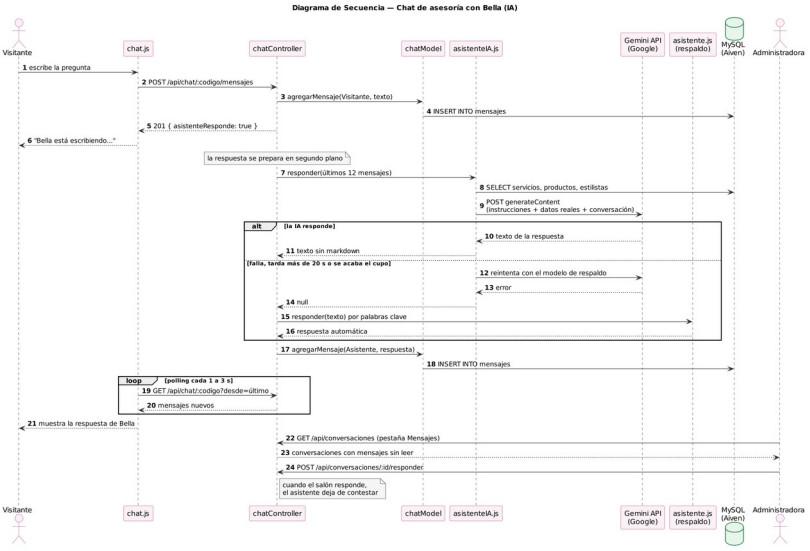



*Figura 3b. Diagrama de secuencia: chat de asesoría con Bella (IA)*

| No | Descripción | Detalle |
|---|---|---|
| | | **Diagrama de componentes.** Los componentes se conectan por **interfaces**: la bola es la interfaz que un componente **provee** y la media luna la que **requiere**. |

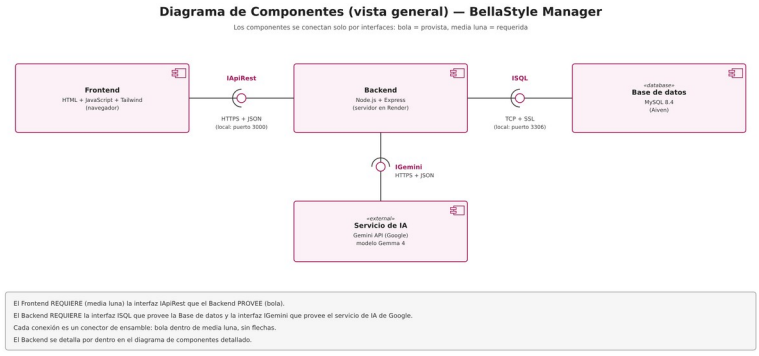



*Figura 4. Componentes: vista general*

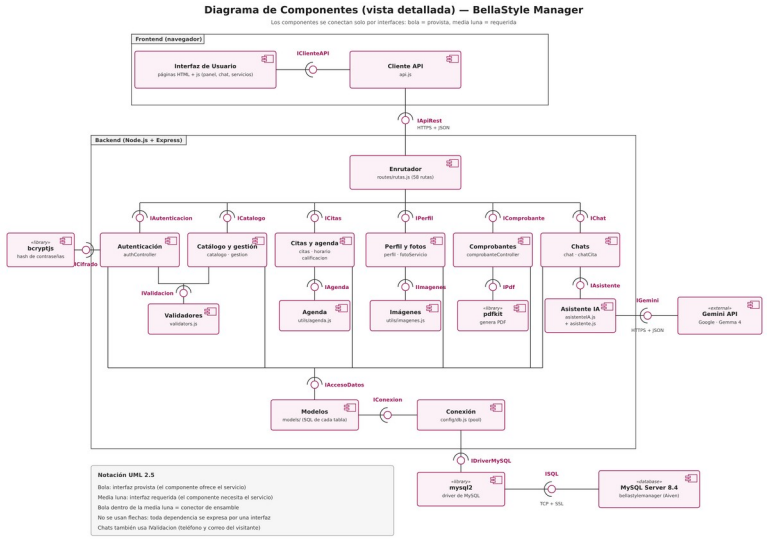



*Figura 5. Componentes: vista detallada*

**Diagrama de despliegue.** Muestra dónde se ejecuta cada parte en producción: el navegador del usuario, el servidor Node.js en **Render**, la base de datos MySQL en **Aiven** y la **API de Gemini** de Google, comunicados por HTTPS y por TCP con SSL. En el computador del desarrollador el mismo sistema corre en `http://localhost:3000` con MySQL local (puerto 3306).

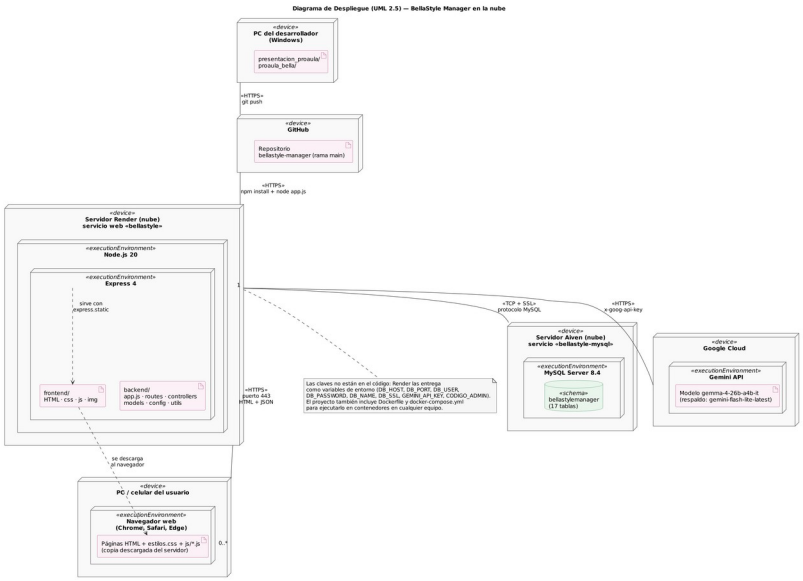
<table>
<tr><th>No</th><th>Descripción</th><th>Detalle</th></tr>
<tr><td></td><td></td><td> 


*Figura 6. Diagrama de despliegue en la nube*</td></tr>
<tr><td>4</td><td>Administración de usuarios</td><td>a. Creación, modificación y retiro de usuarios:<ul><li>Creación: cada persona se registra en <code>register.html</code>, o la administradora la crea en la pestaña Registrar usuario. La cuenta (<code>usuarios</code>) y su perfil (<code>clientas</code> o <code>estilistas</code>) se guardan en una transacción.</li><li>Modificación: la administradora edita los datos de las clientas (incluida la profesión) y los servicios de las estilistas; cada usuario cambia su foto de perfil; la administradora puede cambiar su nombre visible; cualquier usuario cambia su contraseña con ¿Olvidaste tu contraseña?.</li><li>Retiro: las estilistas se desactivan (no reciben citas nuevas) en lugar de borrarse, para conservar el historial.</li></ul>b. Roles y perfiles. Los roles están definidos en la columna <code>usuarios.rol</code>:<table><tr><th>Rol</th><th>Permisos</th></tr><tr><td>Clienta</td><td>Agendar, reprogramar (una vez) y cancelar sus citas; calificarlas, chatear con su estilista y descargar el comprobante PDF; editar sus datos y su foto.</td></tr><tr><td>Estilista</td><td>Ver sus citas; confirmar, finalizar o cancelar; escribir notas del servicio; definir su horario y ausencias; chatear con sus clientas; ver su calificación y su comisión.</td></tr><tr><td>Administradora</td><td>Ver todas las citas e ingresos, confirmarlas o cancelarlas; gestionar servicios, productos (con fotos), estilistas, clientas y cuentas; ver el historial de cada clienta y editar notas; publicar en la vitrina; responder el chat de visitantes. Se registra con el código de administradora</td></tr></table></td></tr>
</table>

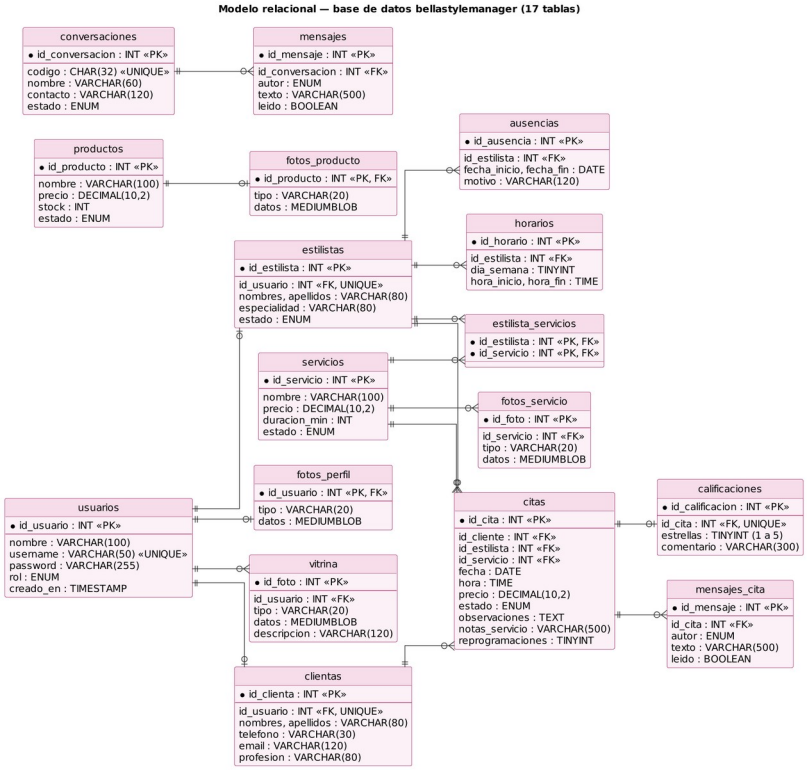
<table>
<tr><th>No</th><th>Descripción</th><th>Detalle</th></tr>
<tr><td></td><td></td><td>
<table>
<tr><td></td><td>(variable de entorno <code>CODIGO_ADMIN</code>).</td></tr>
<tr><td>Visitante</td><td>No tiene cuenta. Puede ver servicios, fotos y productos y escribir en el chat de Bella; se reconoce por el código guardado en su navegador.</td></tr>
</table>
</td></tr>
<tr><td>5</td><td>Modelo relacional de la base de datos</td><td>
Base <code>bellastylemanager</code> en MySQL 8 (<code>utf8mb4</code>) con 17 tablas, creada con <code>init_db.sql</code> y actualizada sin borrar datos con <code>actualizar_bd.js</code>:


*Figura 7. Modelo relacional*

**Diccionario de datos:**

| Tabla | Columna | Tipo | Descripción |
|---|---|---|---|
| usuarios | id_usuario | INT PK | Identificador de la cuenta. |
| | username | VARCHAR(50) UNIQUE | Usuario para iniciar sesión. |
| | password | VARCHAR(255) | Hash bcrypt de la contraseña. |
| | rol | ENUM | Administradora, Estilista o Clienta. |
| clientas | id_clienta | INT PK | Identificador de la clienta. |
</td></tr>
</table>

| No | Descripción | Detalle | | | |
|---|---|---|---|---|---|
| | | | id_usuario | INT FK UNIQUE | Cuenta asociada (puede ser nula). |
| | | | nombres, apellidos, telefono, email, profesion | VARCHAR | Datos de contacto y ocupación de la clienta. |
| | | estilistas | id_estilista | INT PK | Identificador de la estilista. |
| | | | id_usuario | INT FK UNIQUE | Cuenta asociada. |
| | | | estado | ENUM | Activo o Inactivo. |
| | | servicios | id_servicio | INT PK | Identificador del servicio. |
| | | | precio / duracion_min | DECIMAL / INT | Precio y duración en minutos. |
| | | | estado | ENUM | Disponible o No disponible. |
| | | estilista_servicios | id_estilista, id_servicio | INT PK FK | Qué servicios sabe hacer cada estilista (N a M). |
| | | citas | id_cita | INT PK | Identificador de la cita. |
| | | | id_cliente, id_estilista, id_servicio | INT FK | Clienta, estilista y servicio. |
| | | | fecha / hora | DATE / TIME | Inicio de la cita. |
| | | | precio | DECIMAL(10,2) | Precio del día en que se agendó. |
| | | | estado | ENUM | Pendiente, Confirmada, Finalizada o Cancelada. |
| | | | notas_servicio | VARCHAR(500) | Lo que hizo la estilista (productos, tono, técnica). |
| | | | reprogramaciones | TINYINT | Veces que se reprogramó (máximo 1). |
| | | productos | id_producto | INT PK | Identificador del producto. |
| | | | precio / stock / estado | DECIMAL / INT / ENUM | Precio, unidades y Activo/Inactivo. |
| | | conversacion | id_conversacion | INT PK | Identificador de la |

| No | Descripción | Detalle | | | |
|---|---|---|---|---|---|
| | | es | | | conversación del chat. |
| | | | codigo | CHAR(32) UNIQUE | Código aleatorio que el navegador del visitante guarda para reconocerlo sin cuenta. |
| | | | nombre / contacto | VARCHAR | Nombre y teléfono o correo (opcional). |
| | | | estado | ENUM | Abierta o Cerrada. |
| | | mensajes | id_mensaje | INT PK | Identificador del mensaje. |
| | | | id_conversacion | INT FK | Conversación a la que pertenece (ON DELETE CASCADE). |
| | | | autor | ENUM | Visitante, Salon o Asistente. |
| | | | texto / leido | VARCHAR(500) / BOOLEAN | Contenido y si la administradora ya lo vio. |
| | | horarios | id_horario | INT PK | Identificador del horario. |
| | | | id_estilista, dia_semana | INT FK / TINYINT | Un solo horario por estilista y día (UNIQUE). |
| | | | hora_inicio / hora_fin | TIME | Jornada de ese día. |
| | | ausencias | id_ausencia | INT PK | Identificador de la ausencia. |
| | | | id_estilista | INT FK | Estilista ausente (ON DELETE CASCADE). |
| | | | fecha_inicio / fecha_fin / motivo | DATE / VARCHAR(120) | Días en que no atiende y por qué. |
| | | calificaciones | id_calificacion | INT PK | Identificador de la calificación. |
| | | | id_cita | INT FK UNIQUE | Cada cita se califica una sola vez. |
| | | | estrellas / comentario | TINYINT / VARCHAR(300) | De 1 a 5 (CHECK) y comentario opcional. |
| | | mensajes_cita | id_mensaje | INT PK | Identificador del mensaje. |

<table>
<tr><th>No</th><th>Descripción</th><th>Detalle</th></tr>
<tr><td></td><td></td><td>
<table>
<tr><td></td><td>id_cita / autor</td><td>INT FK / ENUM</td><td>Cita del chat; autor Clienta o Estilista.</td></tr>
<tr><td></td><td>texto / leido</td><td>VARCHAR(500) / BOOLEAN</td><td>Contenido y si ya se leyó.</td></tr>
<tr><td>fotos_servicio</td><td>id_foto / id_servicio</td><td>INT PK / INT FK</td><td>Hasta 6 fotos por servicio.</td></tr>
<tr><td></td><td>tipo / datos</td><td>VARCHAR(20) / MEDIUMBLOB</td><td>Formato (webp, png, jpg) e imagen.</td></tr>
<tr><td>fotos_producto</td><td>id_producto</td><td>INT PK FK</td><td>Una foto por producto.</td></tr>
<tr><td></td><td>tipo / datos</td><td>VARCHAR(20) / MEDIUMBLOB</td><td>Formato e imagen.</td></tr>
<tr><td>fotos_perfil</td><td>id_usuario</td><td>INT PK FK</td><td>Una foto de perfil por usuario.</td></tr>
<tr><td></td><td>tipo / datos</td><td>VARCHAR(20) / MEDIUMBLOB</td><td>Formato e imagen.</td></tr>
<tr><td>vitrina</td><td>id_foto / id_usuario</td><td>INT PK / INT FK</td><td>Foto de trabajo y quién la publicó.</td></tr>
<tr><td></td><td>datos / descripcion</td><td>MEDIUMBLOB / VARCHAR(120)</td><td>Imagen y texto que la acompaña.</td></tr>
</table>
Las imágenes se guardan en la base de datos (<code>MEDIUMBLOB</code>) porque el disco del servidor gratuito de Render se borra en cada reinicio. Antes de subirlas, el navegador las reduce a WEBP de máximo 1.200 px y el servidor valida su tipo real y que no pasen de 1,5 MB.
</td></tr>
<tr><td>6</td><td>Descripción de la plataforma</td><td>
BellaStyle Manager es una aplicación web cliente-servidor. El servidor está hecho en Node.js con Express y la base de datos es MySQL. La interfaz está desarrollada en HTML5, JavaScript y Tailwind CSS. En producción el servidor corre en Render, la base de datos en Aiven y el asistente usa la API de Gemini de Google.
<table>
<tr><th>Herramienta</th><th>Versión</th><th>Uso</th></tr>
<tr><td>Node.js</td><td>18 o superior</td><td>Entorno de ejecución del servidor.</td></tr>
<tr><td>Express</td><td>4.x</td><td>Rutas de la API y archivos estáticos.</td></tr>
<tr><td>MySQL Server</td><td>8.x</td><td>Sistema manejador de base de datos (SGBD).</td></tr>
<tr><td>mysql2</td><td>3.x</td><td>Librería de conexión a MySQL (pool).</td></tr>
</table>
</td></tr>
</table>

| No | Descripción | Detalle |
|---|---|---|

| | | |
|---|---|---|
| bcryptjs | 2.4 | Hash de contraseñas. |
| Tailwind CSS | 3.4 | Estilos, compilados con su CLI. |
| pdfkit | 0.15 | Generación del comprobante PDF. |
| Gemini API | v1beta | Inteligencia artificial del chat (modelo gemma-4-26b-a4b-it; respaldo gemini-flash-lite-latest). |
| Render | Plan gratuito | Servidor web en la nube (ejecuta node app.js). |
| Aiven for MySQL | 8.4 | Base de datos administrada en la nube, con SSL. |
| Docker y Docker Compose | imagen node:20-alpine | Contenedores para ejecutar el sistema en cualquier equipo. |
| Git y GitHub | — | Control de versiones y repositorio del equipo. |

**Para instalar el servidor se requiere:**

1. Windows 10/11 (también funciona en Linux o macOS), 4 GB de RAM y 1 GB libre en disco.

2. Node.js 18 o superior y MySQL Server 8.

3. Ejecutar `INSTALAR_BASE_DE_DATOS.bat` (crea la base y los datos de prueba; si ya existe, la borra y la crea de nuevo). Para agregar tablas y columnas nuevas sin perder datos, ejecutar `node actualizar_bd.js` en la carpeta `backend`.

4. Revisar usuario y contraseña de MySQL en `backend/config/config.js`, o definirlos en el archivo `backend/.env` (`DB_USER`, `DB_PASSWORD`), donde también va la clave `GEMINI_API_KEY`. Este archivo no se sube a GitHub.

5. Ejecutar `backend/INICIAR_SERVIDOR.bat` y abrir `http://localhost:3000`.

**Para el mantenimiento se requiere:** Visual Studio Code (o similar), MySQL Workbench y npm. Si se cambian clases de Tailwind, ejecutar `npm run css` en la carpeta `backend`.

**Despliegue en la nube:**

1. Subir los cambios a GitHub (`git push` a la rama `main`).

2. Render descarga el código, ejecuta `npm install` y arranca `node app.js` (carpeta raíz `proaula_bella/backend`).

3. Variables de entorno en Render: `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `DB_SSL=true`, `GEMINI_API_KEY` y `CODIGO_ADMIN`.

4. Cuando cambia la estructura de la base, ejecutar `node`

<table>
<tr><th>No</th><th>Descripción</th><th>Detalle</th></tr>
<tr><td></td><td></td><td><code>actualizar_bd.js</code> con las variables de Aiven.<br>Alternativa con contenedores: <code>docker compose up --build</code> levanta MySQL y la aplicación juntos.</td></tr>
<tr><td>7</td><td>Documentación del código fuente</td><td>1. Arquitectura. El backend está organizado en capas: rutas → controladores → modelos → base de datos. Las rutas reciben la petición, los controladores aplican las reglas de negocio y solo los modelos ejecutan SQL.<br>2. Estructura de directorios:
<table>
<tr><th>Directorio</th><th>Descripción</th></tr>
<tr><td>backend/</td><td>Servidor Node.js. <code>app.js</code> es el arrancador de la aplicación.</td></tr>
<tr><td>backend/config</td><td>Configuración (puerto, MySQL, código admin, clave de IA) y pool de conexiones.</td></tr>
<tr><td>backend/routes</td><td>Lista de rutas (endpoints) de la API.</td></tr>
<tr><td>backend/controllers</td><td>Lógica y reglas de negocio.</td></tr>
<tr><td>backend/models</td><td>Consultas SQL de cada tabla.</td></tr>
<tr><td>backend/utils</td><td>Validaciones, funciones de agenda, imágenes y asistentes del chat (IA y palabras clave).</td></tr>
<tr><td>frontend/</td><td>Páginas HTML que ve el usuario.</td></tr>
<tr><td>frontend/js</td><td>Lógica del navegador y llamadas a la API.</td></tr>
<tr><td>frontend/src y css</td><td>Estilos de Tailwind: fuente y archivo compilado.</td></tr>
<tr><td>frontend/img y fonts</td><td>Imágenes y fuentes locales.</td></tr>
</table>
3. Relación de programas:
<table>
<tr><th>Programa</th><th>Lenguaje</th><th>Descripción</th></tr>
<tr><td>backend/app.js</td><td>JavaScript</td><td>Arranca Express, sirve el frontend y monta la API en /api.</td></tr>
<tr><td>config/config.js · db.js</td><td>JavaScript</td><td>Datos de conexión y pool de MySQL.</td></tr>
<tr><td>routes/rutas.js</td><td>JavaScript</td><td>Asocia cada URL con su controlador.</td></tr>
<tr><td>controllers/authController.js</td><td>JavaScript</td><td>Login, registro y recuperar contraseña.</td></tr>
<tr><td>controllers/citasController.js</td><td>JavaScript</td><td>Citas: horas libres, reglas de horario, asignación automática, reprogramar una vez, notas e historial.</td></tr>
<tr><td>controllers/</td><td>JavaScript</td><td>Consultas de servicios,</td></tr>
</table></td></tr>
</table>

| No | Descripción | Detalle |
|---|---|---|

| Programa | Lenguaje | Descripción |
|---|---|---|
| catalogoController.js | | productos, estilistas y clientas. |
| controllers/ gestionController.js | JavaScript | Crear, editar, activar y desactivar. |
| models/*.js (12 archivos) | JavaScript / SQL | Un módulo de acceso a datos por tabla o tema. |
| utils/validators.js | JavaScript | Validación de nombre, usuario, contraseña, teléfono (10 dígitos) y correo. |
| controllers/chatController.js | JavaScript | Chat: iniciar conversación, enviar, leer, responder y cerrar. |
| models/chatModel.js | JavaScript / SQL | Consultas de las tablas conversaciones y mensajes. |
| utils/asistente.js | JavaScript | Respuestas automáticas por palabras clave (respaldo de la IA). |
| instalar_bd.js | JavaScript | Ejecuta init_db.sql. |
| init_db.sql | SQL | Crea las tablas y los datos de prueba. |
| welcome · index · register · recuperar · panel (.html) | HTML | Pantallas. index.html (inicio de sesión, “Cómo funciona” y menú de servicios) es la primera: la ruta / redirige ahí. |
| js/api.js | JavaScript | Función `pedir()` (fetch) y manejo de sesión. |
| js/login.js · registro.js · inicio.js | JavaScript | Lógica del inicio de sesión, del registro (con profesión) y de la presentación (vitrina y productos). |
| js/panel.js | JavaScript | Paneles de los tres roles: citas, horario, fotos, vitrina, historial, mensajes y comprobante. |
| js/chat.js | JavaScript | Burbuja del chat de Bella e invitación a los 10 segundos; se dibuja sola en las páginas que incluyen este archivo. |
| src/estilos.css | CSS (Tailwind) | Componentes de estilo reutilizables. |
| controllers/ horarioController.js | JavaScript | Horario semanal y ausencias de cada estilista. |
| controllers/ | JavaScript | Calificar citas finalizadas y |

| No | Descripción | Detalle |
|---|---|---|

| Programa | Lenguaje | Descripción |
|---|---|---|
| calificacionController.js | | promedio por estilista. |
| controllers/ chatCitaController.js | JavaScript | Chat entre la clienta y la estilista de una cita. |
| controllers/ comprobanteController.js | JavaScript | Comprobante PDF del servicio (pdfkit). |
| controllers/perfilController.js · fotoServicioController.js | JavaScript | Foto de perfil, nombre, vitrina y fotos de servicios y productos. |
| utils/asistenteIA.js | JavaScript | Asistente Bella: arma las instrucciones con datos reales y llama a la API de Gemini. |
| utils/agenda.js | JavaScript | Funciones de tiempo: horas a minutos, cruces de horario, hora de Colombia. |
| utils/imagenes.js | JavaScript | Valida imágenes (tipo real y máximo 1,5 MB) y las envía. |
| actualizar_bd.js | JavaScript | Crea tablas y columnas nuevas sin borrar datos. |
| js/servicios.js · tema.js | JavaScript | Menú de servicios con fotos; modo claro/oscuro y videos de fondo. |
| Dockerfile · docker-compose.yml | Docker | Imagen de la app y contenedores de app + MySQL. |

**4. API REST.** Todas las rutas (58) empiezan por `/api` y responden JSON `{ success, message, data }`; las imágenes y el PDF se envían como archivo:

| Método | Ruta | Descripción |
|---|---|---|
| POST | /login · /register · /recuperar-password | Cuentas. |
| GET / POST | /citas | Listar y crear citas. |
| PUT | /citas/:id/estado · /reprogramar · /notas | Cambiar estado, reprogramar (una vez) y guardar notas. |
| GET | /servicios · /productos · /estilistas · /clientas | Consultas. |
| POST / PUT | /servicios · /productos · /clientas/:id | Crear y editar. |
| PUT | /.../:id/estado · /estilistas/:id/servicios | Activar, desactivar, asignar. |

| No | Descripción | Detalle |
|---|---|---|

| Método | Ruta | Descripción |
|---|---|---|
| POST | /chat | Inicia una conversación (visitante). |
| GET / POST | /chat/:codigo · /chat/:codigo/mensajes | Leer mensajes nuevos y enviar (visitante). |
| GET | /conversaciones · /conversaciones/:id/mensajes | Lista y lectura (administradora). |
| POST / PUT | /conversaciones/:id/responder · /:id/estado | Responder, cerrar o reabrir. |
| GET | /disponibilidad | Horas libres de un servicio en una fecha. |
| POST | /citas/:id/calificacion | Calificar una cita finalizada. |
| GET / POST | /citas/:id/mensajes | Chat clienta–estilista. |
| GET | /citas/:id/comprobante · /clientas/:id/historial | Comprobante PDF e historial de servicios. |
| GET / PUT / POST / DELETE | /estilistas/:id/horario · /estilistas/:id/ausencias · /ausencias/:id | Horario y ausencias. |
| GET / POST / DELETE | /fotos-servicios · /servicios/:id/fotos · /fotos/:id | Fotos de servicios. |
| GET / PUT / DELETE | /usuarios/:id/foto · /productos/:id/foto | Foto de perfil y de producto. |
| PUT | /usuarios/:id/nombre | Cambiar el nombre visible. |
| GET / POST / PUT / DELETE | /vitrina · /vitrina/:id · /vitrina/:id/imagen | Vitrina de trabajos. |
| GET | /estilistas/:id/calificaciones | Calificaciones de una estilista. |

**5. Reglas principales: horas libres y asignación automática.** Para mostrar las horas libres, el servidor recorre el día de 30 en 30 minutos (de 8:00 a 19:00) y descarta las franjas en que ninguna estilista capacitada puede atender: no trabaja ese día, está ausente, la hora queda fuera de su horario o ya tiene una cita. Al agendar, de las estilistas libres se escoge la que tiene menos citas ese día. Dos horarios se cruzan así:

```
function seCruzan(inicioA, finA, inicioB, finB) {
    return inicioA < finB && inicioB < finA;
}
```

**6. Chat y asistente con IA.** El visitante no inicia sesión: al empezar el chat el servidor genera un código aleatorio de 32 caracteres

<table>
<tr><th>No</th><th>Descripción</th><th>Detalle</th></tr>
<tr><td></td><td></td><td>(<code>crypto.randomBytes</code>) que el navegador guarda en <code>localStorage</code>. Cada mensaje se guarda y el servidor responde de inmediato; en segundo plano, <code>utils/asistenteIA.js</code> consulta en MySQL los servicios, precios, productos, estilistas y horario, y los envía a la API de Gemini (modelo <code>gemma-4-26b-a4b-it</code>, respaldo <code>gemini-flash-lite-latest</code>) junto con las instrucciones de Bella y los últimos 12 mensajes. La clave está en la variable de entorno <code>GEMINI_API_KEY</code> y nunca llega al navegador. Si la IA falla o tarda más de 20 segundos, <code>utils/asistente.js</code> responde por palabras clave. El asistente deja de responder cuando el salón escribe en esa conversación.<br><br>7. Seguridad: contraseñas con hash bcrypt, consultas parametrizadas (evitan inyección SQL), registro en transacción, validación en el servidor (incluido el teléfono de 10 dígitos), imágenes validadas por tipo real y tamaño, textos escapados antes de mostrarlos (evita XSS), claves en variables de entorno y conexiones cifradas (HTTPS y SSL). Mejoras previstas: token de sesión (JWT) en cada petición con permisos por rol, máximo 3 intentos de inicio de sesión y recuperación de contraseña con verificación del teléfono o correo registrado.<br><br>8. Comprobante PDF. <code>comprobanteController.js</code> usa <code>pdfkit</code> para generar un PDF tamaño A5 con el logo, los datos de la clienta, el servicio, la estilista, las notas, la calificación y el total. Solo se genera para citas finalizadas.</td></tr>
<tr><td>8</td><td>Acuerdos de niveles de servicio (ANS)</td><td>Los casos se reciben en la mesa de ayuda y se atienden según su prioridad:<br><br>
<table>
<tr><th>Prioridad</th><th>Ejemplo</th><th>Respuesta</th><th>Solución</th></tr>
<tr><td>Alta</td><td>El servidor no inicia o no hay conexión con MySQL; nadie puede ingresar.</td><td>1 hora</td><td>4 horas</td></tr>
<tr><td>Media</td><td>Una función falla (no se puede agendar, editar o cambiar estados).</td><td>4 horas</td><td>1 día hábil</td></tr>
<tr><td>Baja</td><td>Dudas de uso, cambios de datos, textos o mejoras.</td><td>1 día hábil</td><td>3 días hábiles</td></tr>
</table>
<br>Escalamiento: el primer nivel (administradora del salón) resuelve cuentas, servicios, precios y datos. Si no puede resolverlo, escala al segundo nivel (soporte técnico), que revisa el servidor, la base de datos y el código.</td></tr>
</table>

# Glosario

**Aiven:** proveedor que aloja y administra bases de datos en la nube; aquí aloja el MySQL de BellaStyle.

**ANS:** acuerdo de niveles de servicio. Define tiempos de respuesta y escalamiento de los casos de soporte.

**API key:** clave secreta que identifica al sistema ante un servicio externo, como la API de Gemini.

**API REST:** conjunto de direcciones (rutas) por las que el navegador pide y envía datos al servidor en formato JSON.

**bcrypt:** algoritmo que transforma la contraseña en un hash irreversible, de forma que no se pueda leer.

**Docker:** herramienta que empaqueta la aplicación con su entorno en contenedores que funcionan igual en cualquier equipo.

**Gemini API:** servicio de Google que da acceso a modelos de inteligencia artificial; lo usa el asistente Bella.

**Hash:** transformación de un solo sentido que convierte la contraseña en un texto que no se puede revertir.

**Librería:** conjunto de programas que apoyan un área específica y que el programador reutiliza (por ejemplo, mysql2).

**Polling:** técnica en la que el navegador pregunta al servidor cada pocos segundos si hay datos nuevos. La usa el chat.

**Pool de conexiones:** grupo de conexiones a la base de datos que se reutilizan para responder más rápido.

**Render:** plataforma en la nube que publica el servidor Node.js a partir del repositorio de GitHub.

**SGBD:** sistema manejador de bases de datos; en este proyecto, MySQL.

**UML:** lenguaje unificado de modelado, usado para diseñar sistemas con diagramas.

**Variable de entorno:** valor de configuración guardado fuera del código, como contraseñas y claves.

# Control del documento

| Campo | Detalle |
|---|---|
| Documento | Manual Técnico – BellaStyle Manager |
| Proyecto | Proyecto de Aula · Ingeniería de Sistemas |
| Institución | Fundación Universitaria Colombo Internacional |
| Autor | César Pestana |
| Versión | 4.0 – noviembre de 2026 (17 tablas, asistente con IA, horarios, calificaciones, fotos, comprobante PDF y despliegue en Render y Aiven) |
| Referencia | SENA. Manuales técnicos de la aplicación. IEEE 1063-2001, Standard for Software User Documentation. |

# Anexo A. Diagramas en tamaño completo

Los mismos diagramas de las secciones 3 y 5, en tamaño completo para leerlos con detalle.

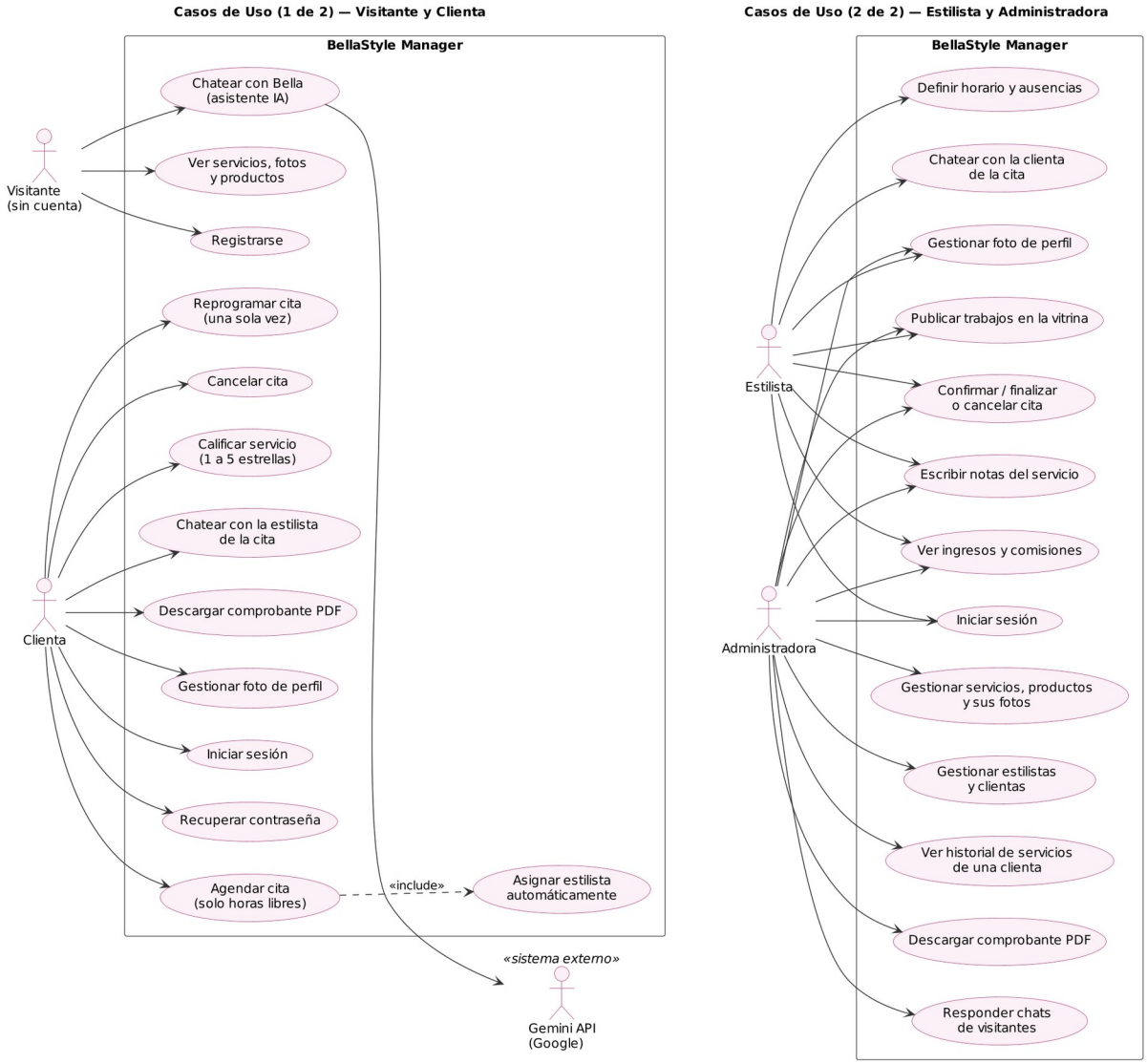



*Figura 1. Diagramas de casos de uso*

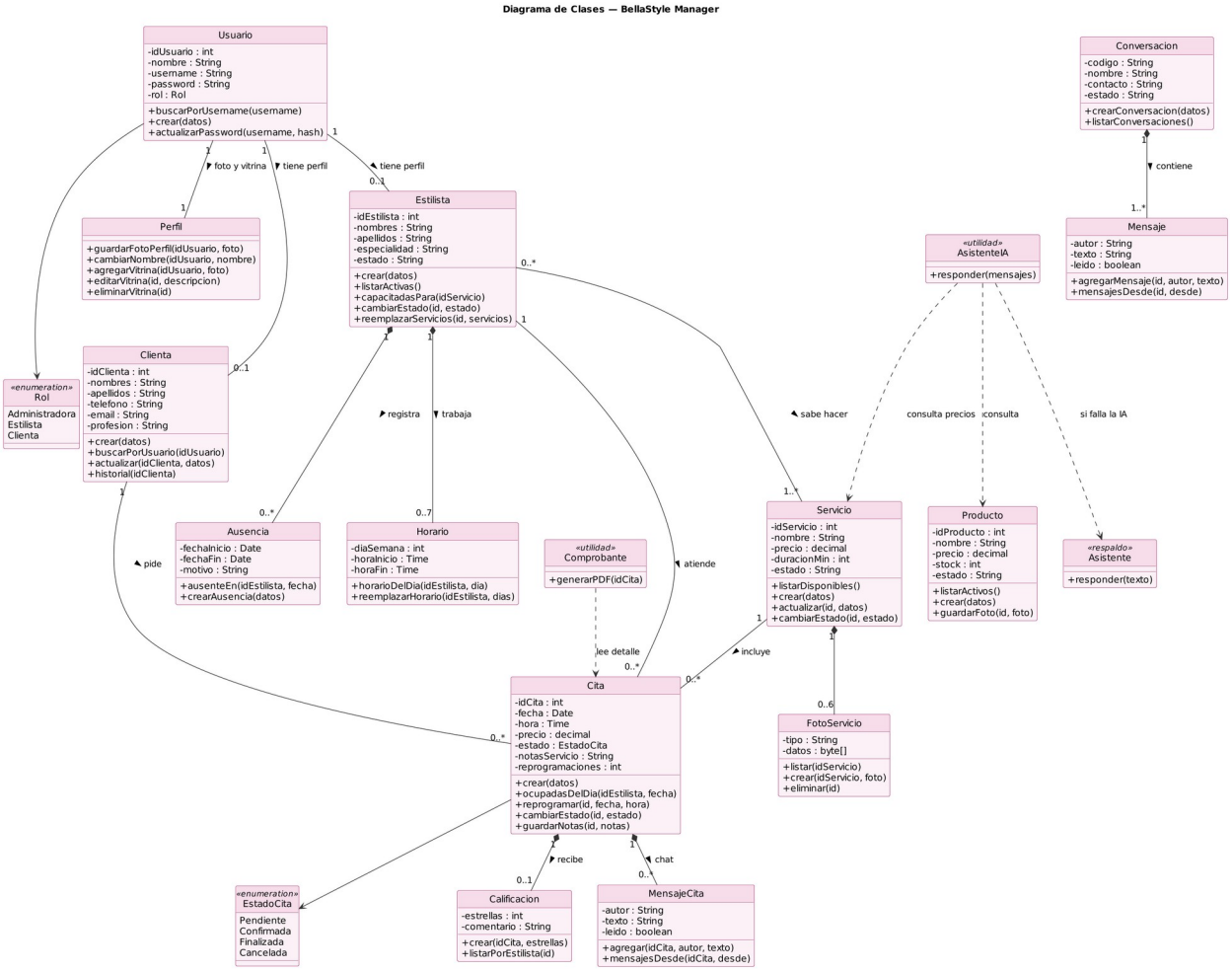



*Figura 2. Diagrama de clases*

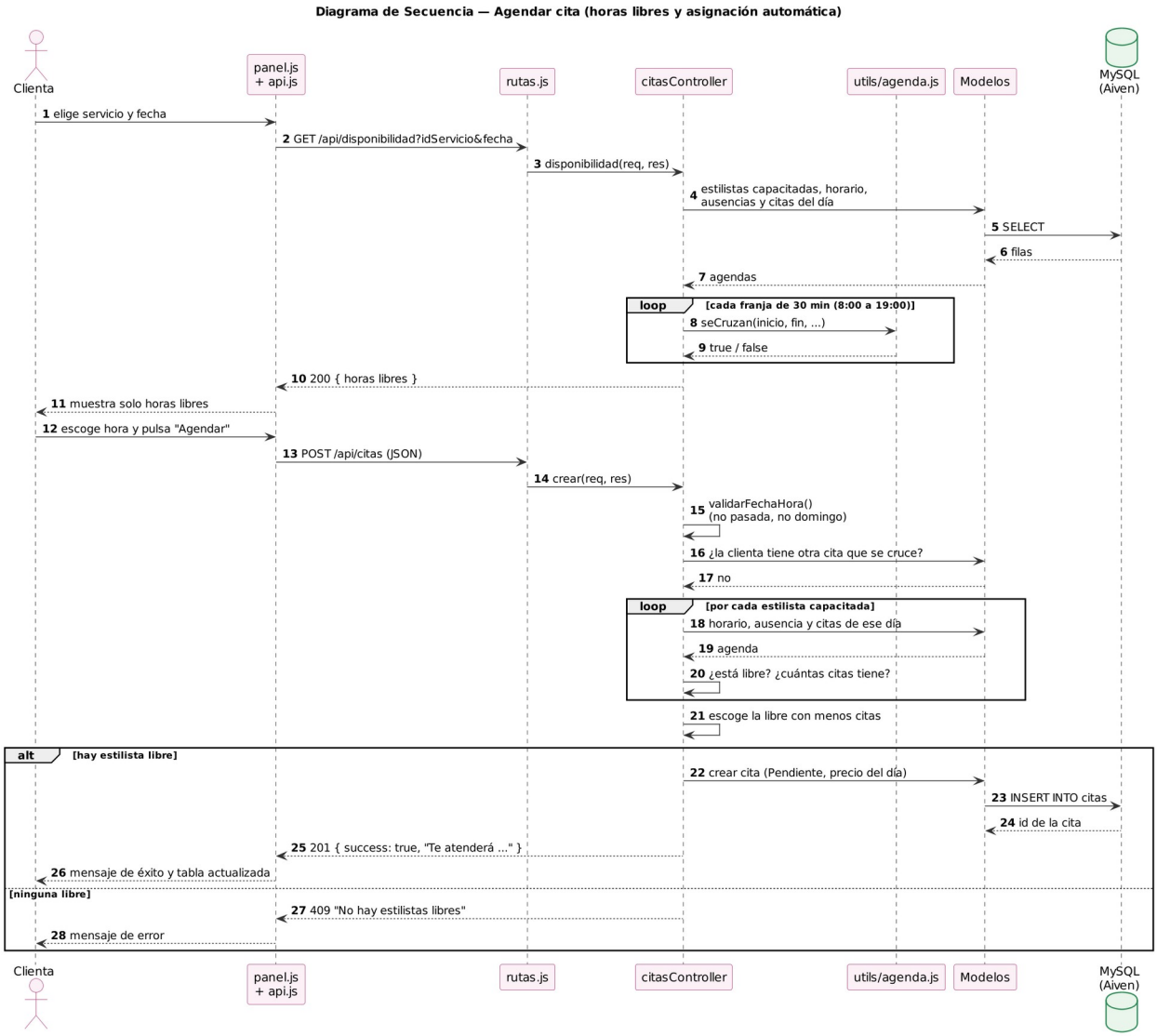



*Figura 3. Diagrama de secuencia: agendar cita*

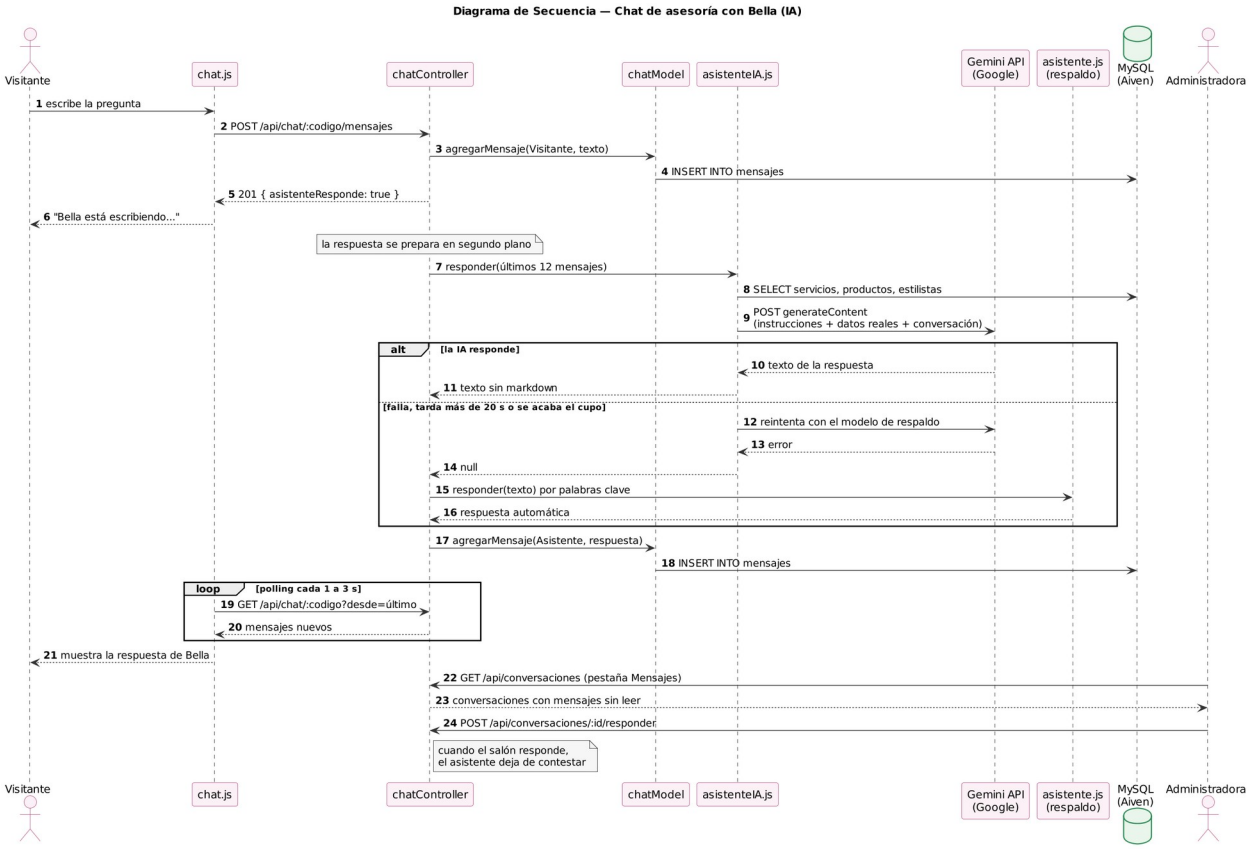



*Figura 3b. Diagrama de secuencia: chat de asesoría con Bella (IA)*

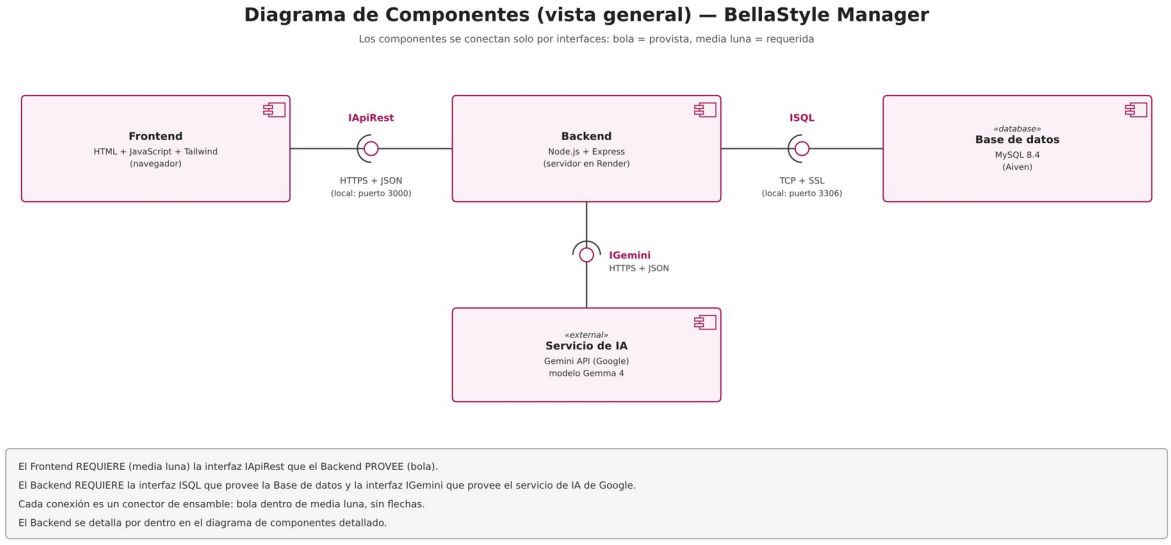



*Figura 4. Componentes: vista general*

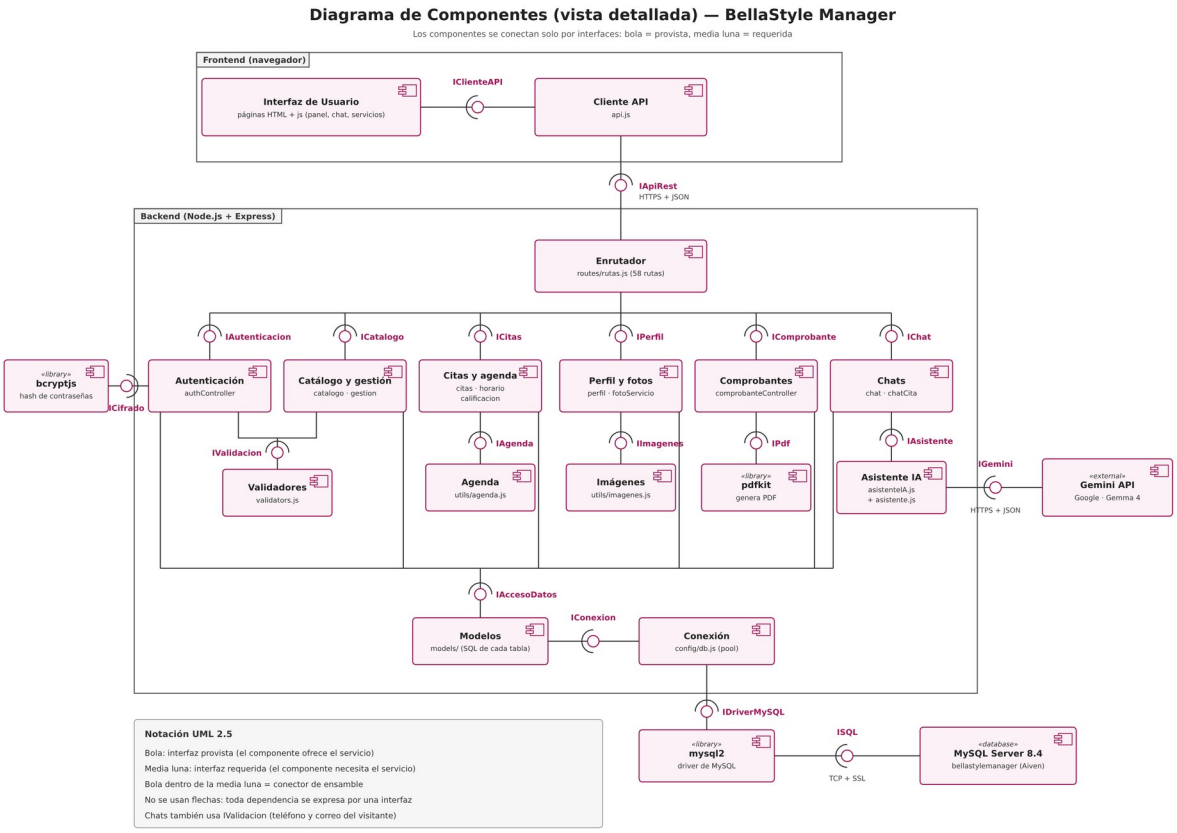



*Figura 5. Componentes: vista detallada*

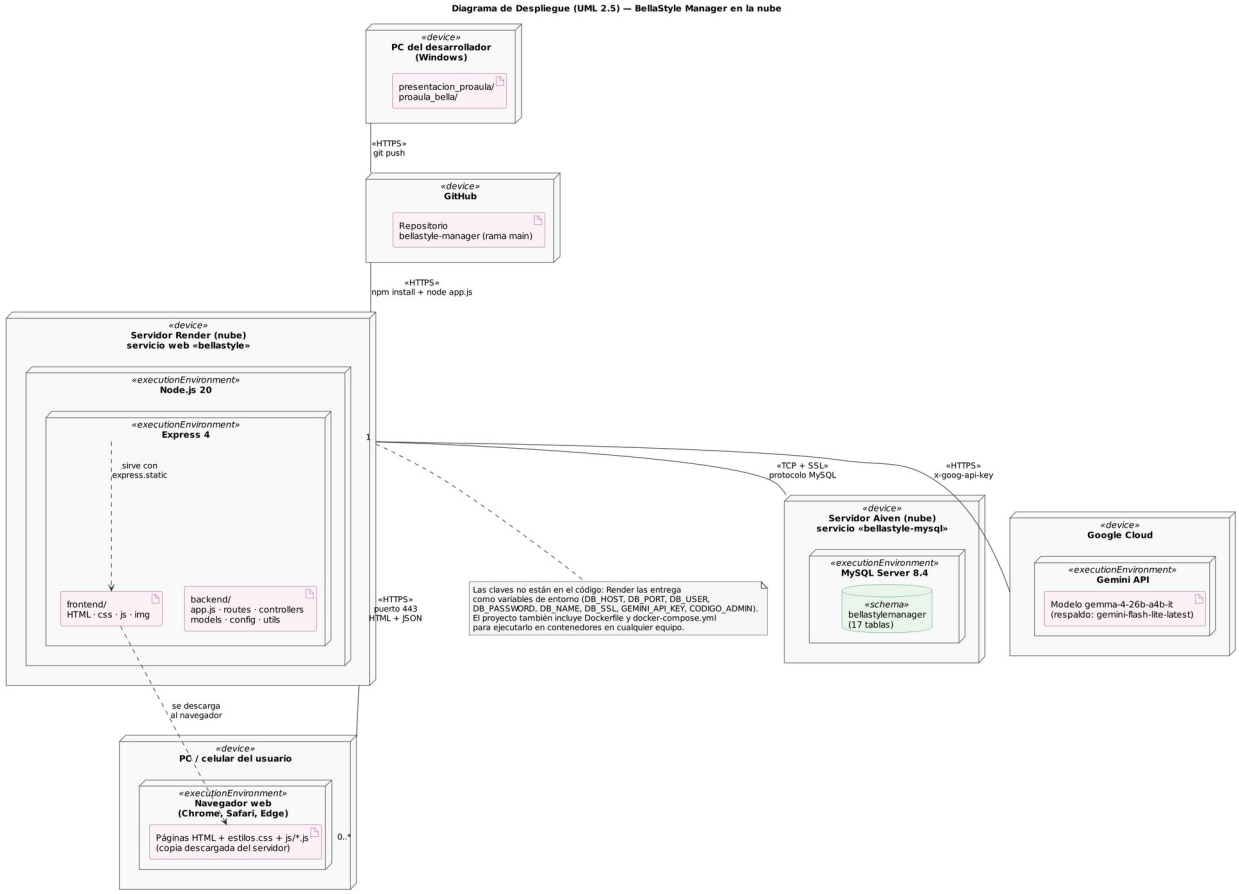



*Figura 6. Diagrama de despliegue en la nube*

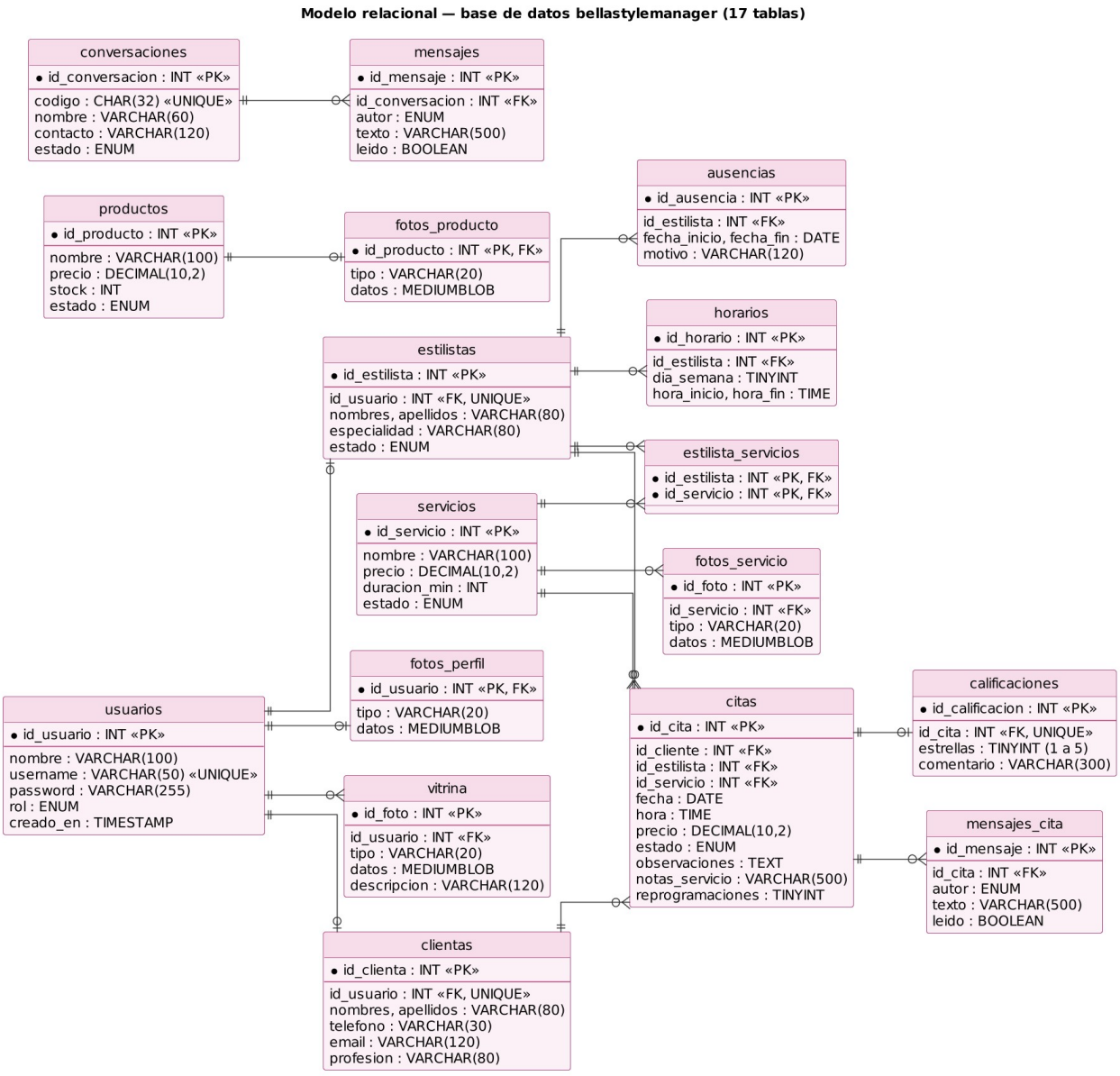



*Figura 7. Modelo relacional*